



Análisis y Desarrollo de Software

Evidencia 3: Avance del Código del Backend Proyecto Memorize Evolutivo

Motor lógico, reglas de negocio, seguridad y APIs del sistema

Jose David Sandoval Caceres
Kevin Gregorio Mora

Ficha 3172526

SERVICIO NACIONAL DE APRENDIZAJE SENA
Atención al sector agropecuario
Piedecuesta - Santander

Este documento presenta la evidencia del código del backend: el enlace al repositorio público donde se aloja el código, fragmentos claros de la lógica implementada (definición de rutas, un controlador, el middleware de autenticación y una regla de negocio) y las capturas de las pruebas realizadas a los endpoints con cliente HTTP, mostrando las respuestas reales en formato JSON.

1. Objetivo y alcance del backend

El backend es el motor lógico de la aplicación: procesa las reglas de negocio (XP, monedas, partidas, salas multijugador), gestiona la seguridad y la autenticación, y expone los endpoints (APIs REST) y eventos de tiempo real (Socket.IO) que el frontend consume. Es la capa invisible para el usuario, pero fundamental para el funcionamiento del sistema.

Capa	Tecnología / version
Servidor HTTP	Express 5.2 (puerto 5175)
Tiempo real	Socket.IO 4.8 (20+ eventos)
ORM + Base de datos	Prisma 5.22 + PostgreSQL 18
Lenguaje	TypeScript (arquitectura POO por capas)
Capas	routes -> controllers -> services -> repositories -> models
Patrones	Singleton (Container, RoomManager), Strategy + Factory (modos), Repository, DI

Endpoints principales expuestos por el backend:

Metodo	Endpoint	Funcion
GET	/health	Estado del servidor y salas activas
POST	/api/auth/login	Login o registro automatico por email
GET	/api/users/:userId	Perfil, nivel, XP y moneda del jugador
POST	/api/matches	Guardar resultado + XP/monedas automaticos
GET/POST	/api/rooms (+ /:id/join)	Salas multijugador en memoria + persistencia
GET	/api/leaderboard?type=xp	Ranking global, por modo, semanal y mensual
GET/PUT/D ELETE	/api/admin/* (23 endpoints)	Panel admin: usuarios, baneos, anuncios, stats

2. Enlace al repositorio

El código del backend se aloja en el repositorio público de GitHub del proyecto, carpeta **backend/**, rama **main**:

<https://github.com/kevinmora18/memorize>

El repositorio es público e incluye el servidor Express (**backend/src/** con controladores, servicios,

repositorios y modelos POO), el esquema Prisma (**backend/prisma/schema.prisma** con 7 tablas), las pruebas y la configuración de despliegue.

3. Fragmentos de código: lógica implementada

Definición de rutas (las rutas solo declaran endpoints y delegan al controlador):

backend/src/routes/auth.ts (extracto real)

```
import { Router } from 'express';
import { Container } from '../Container';

const router = Router();
const container = Container.getInstance();
const authController = container.authController;

// POST /api/auth/login - Login o registro automatico
router.post('/login', authController.login);

// GET /api/auth/validate/:userId - Validar acceso de usuario
router.get('/validate/:userId', authController.validateAccess);

// GET /api/auth/is-admin/:userId - Verificar si es admin
router.get('/is-admin/:userId', authController.checkAdmin);

export default router;
```

Controlador: recibe HTTP, valida entrada y responde JSON (sin lógica de negocio):

backend/src/controllers/AuthController.ts - login (extracto real)

```
login = async (req: Request, res: Response, next: NextFunction): Promise<void> => {
  try {
    const { email } = req.body;
    if (!email || !email.includes('@')) {
      res.status(400).json({ error: 'Email valido requerido' });
      return;
    }
    const user = await this.authService.loginOrRegister(email);
    res.json(user.toJSON());
  } catch (error: any) {
    if (error.message.includes('baneado')) {
      res.status(403).json({ error: error.message });
    } else { next(error); }
  }
};
```

Middleware de autenticación: protege las rutas de administración por rol:

backend/src/routes/admin.ts - requireAdmin (extracto real)

```
// Middleware para verificar rol de admin
const requireAdmin = async (req: any, res: any, next: any) => {
  try {
    const { adminId } = req.body;
    if (!adminId) {
      return res.status(401).json({ error: 'ID de administrador requerido' });
    }
    const user = await prisma.user.findUnique({ where: { id: adminId } });
    if (!user || user.role !== 'admin') {
      return res.status(403).json({ error: 'Acceso denegado: se requiere rol de administrador' });
    }
    req.admin = user;
    next();
  } catch (error) { ... }
};
// Uso: router.put('/users/:id/currency', requireAdmin, ...);
```

Regla de negocio: login o registro automatico con verificacion de baneo:

backend/src/services/AuthService.ts - loginOrRegister (extracto real)

```
async loginOrRegister(email: string): Promise<User> {
  let user = await this.userRepository.findByEmail(email);
  if (!user) {
    user = await this.userRepository.create({ email }); // nuevo: 500 monedas, 50 gemas
  }
  if (user.isCurrentlyBanned()) {
    throw new Error(user.bannedUntil ? `Usuario baneado hasta ${user.bannedUntil}`
      : 'Usuario baneado permanentemente');
  }
  await this.userRepository.updateLastLogin(user.id);
  return user;
}
```

4. Capturas de funcionamiento (endpoints probados)

Pruebas ejecutadas el 5 de octubre de 2026 contra el backend real corriendo en **http://localhost:5175** (Express + Socket.IO + PostgreSQL, base de datos **memorize**), usando cliente HTTP estilo Thunder Client / Postman. Todas las respuestas JSON mostradas son reales, devueltas en vivo por el servidor.

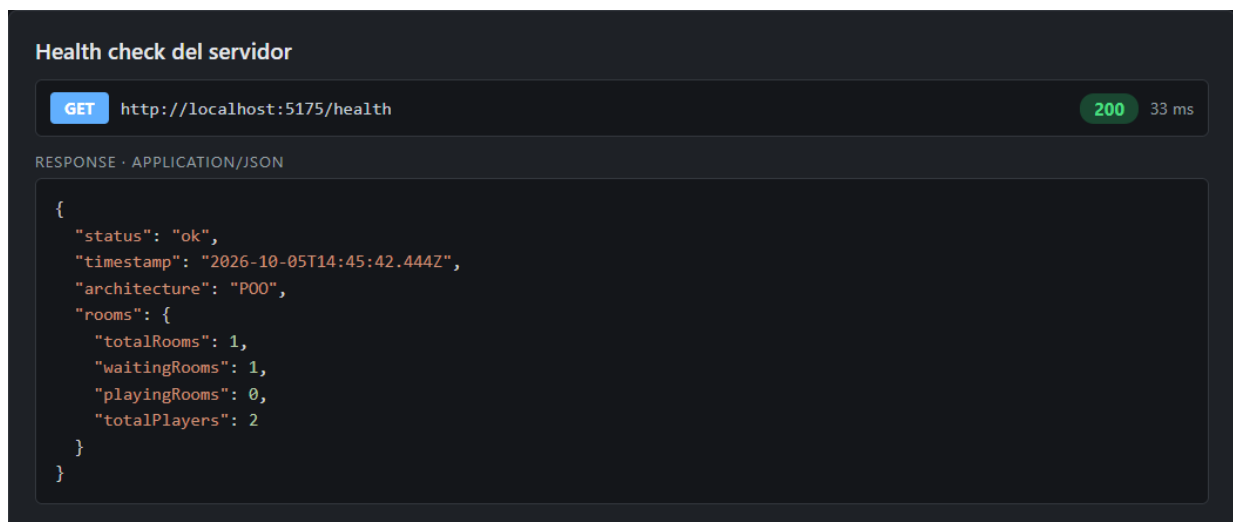


Figura 1 - GET /health (200).

Health check: servidor ok, arquitectura POO y conteo de salas en memoria.

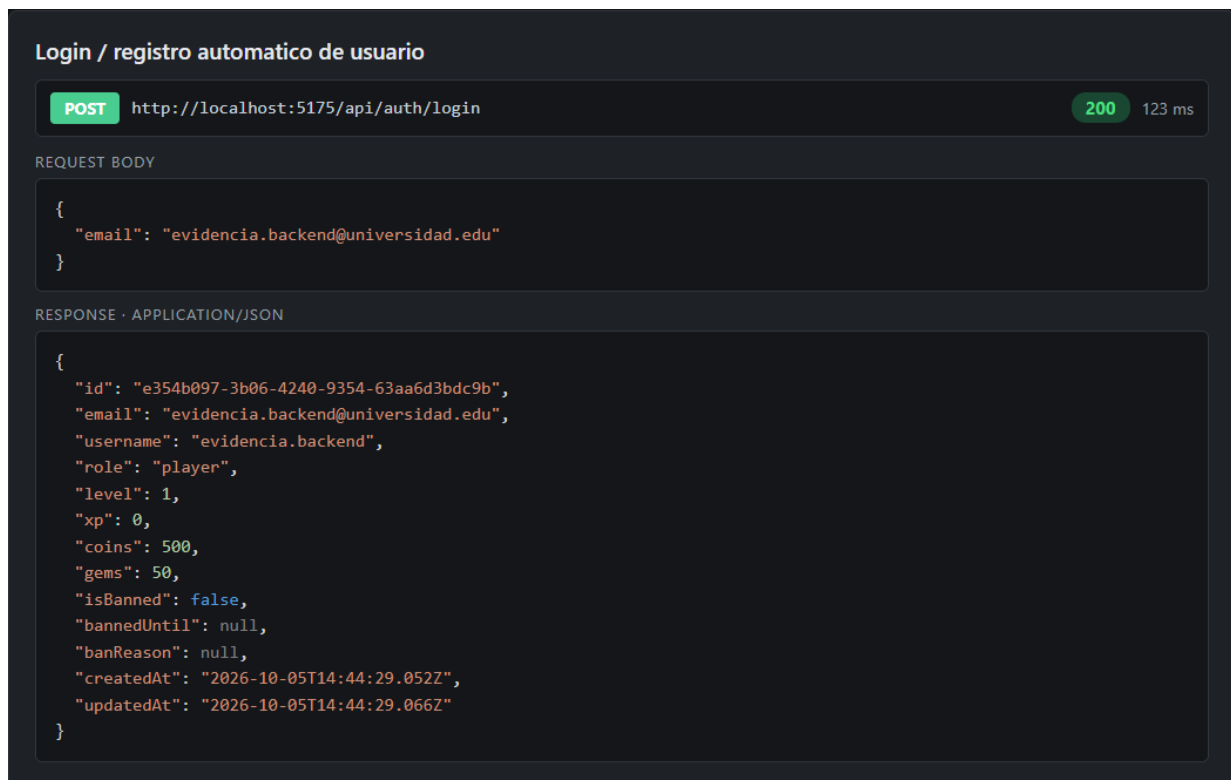


Figura 2 - POST /api/auth/login (200).

Login con email: el servicio crea el usuario automaticamente con 500 monedas y 50 gemas iniciales, rol player y nivel 1, y devuelve el perfil en JSON.

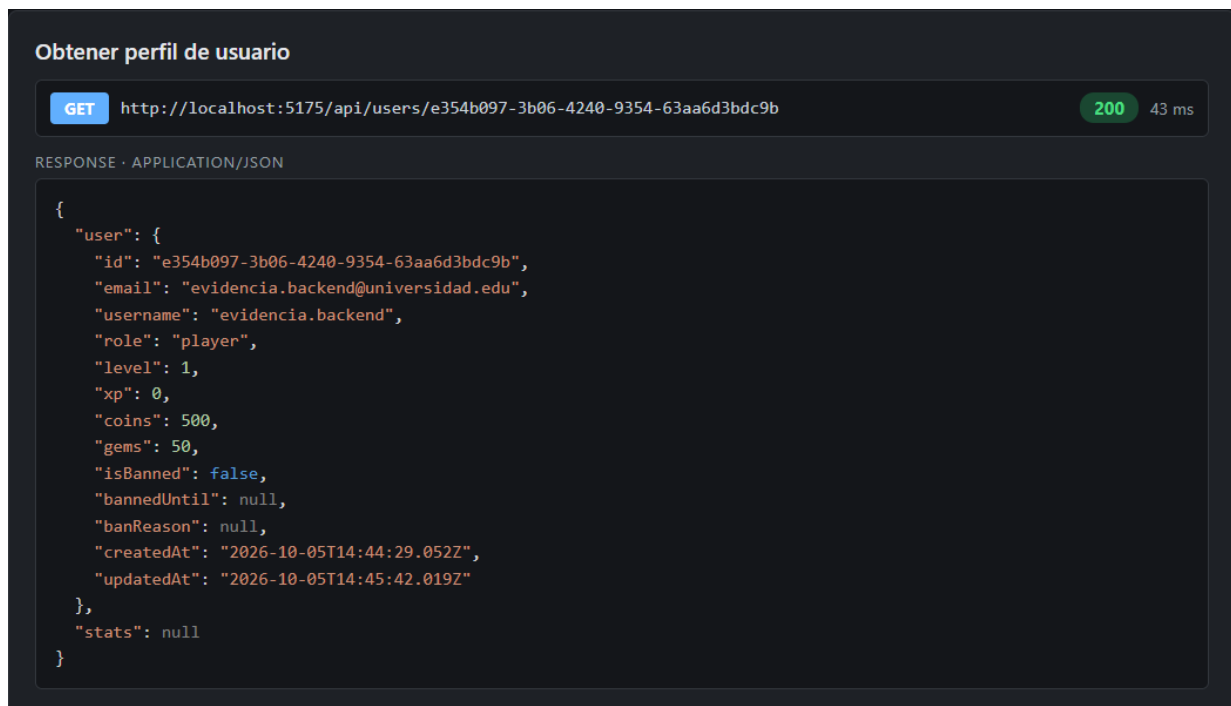


Figura 3 - GET /api/users/:id (200).

Perfil del jugador con nivel, XP, monedas, gemas y estado de baneo.

Crear sala multijugador

POST http://localhost:5175/api/rooms

200 38 ms

REQUEST BODY

```
{
  "name": "Sala Evidencia 3",
  "hostId": "e354b097-3b06-4240-9354-63aa6d3bdc9b",
  "maxPlayers": 4
}
```

RESPONSE · APPLICATION/JSON

```
{
  "id": "room_1791211542213_2ta6fff0q",
  "name": "Sala Evidencia 3",
  "hostId": "e354b097-3b06-4240-9354-63aa6d3bdc9b",
  "hostName": "evidencia.backend",
  "maxPlayers": 4,
  "currentPlayers": 1,
  "mode": "classic",
  "difficulty": "normal",
  "isPrivate": false,
  "status": "waiting",
  "createdAt": "2026-10-05T14:45:42.213Z",
  "players": [
    {
      "id": "e354b097-3b06-4240-9354-63aa6d3bdc9b",
      "socketId": "",
      "name": "evidencia.backend",
      "level": 1,
      "isReady": false,
      "score": 0,
      "matches": 0,
      "isConnected": true
    }
  ],
  "gameState": {
    "status": "waiting",
    "currentRound": 0,
    "totalRounds": 1,
    "cards": [],
    "flippedCards": [],
    "matchedCards": [],
    "currentTurn": null,
    "scores": {}
  }
}
```

Figura 4 - POST /api/rooms (200).

Creacion de sala multijugador: id, codigo, host, capacidad, modo y estado waiting con el jugador anfitrión ya dentro.

Unirse a una sala

POST http://localhost:5175/api/rooms/room_1791211542213_2ta6fff0q/join

200 43 ms

REQUEST BODY

```
{
  "userId": "6dbca2da-dcaf-4741-8efc-60de2f9eddd4"
}
```

RESPONSE · APPLICATION/JSON

```
{
  "id": "room_1791211542213_2ta6fff0q",
  "name": "Sala Evidencia 3",
  "hostId": "e354b097-3b06-4240-9354-63aa6d3bdc9b",
  "hostName": "evidencia.backend",
  "maxPlayers": 4,
  "currentPlayers": 2,
  "mode": "classic",
  "difficulty": "normal",
  "isPrivate": false,
  "status": "waiting",
  "createdAt": "2026-10-05T14:45:42.213Z",
  "players": [
    {
      "id": "e354b097-3b06-4240-9354-63aa6d3bdc9b",
      "socketId": "",
      "name": "evidencia.backend",
      "level": 1,
      "isReady": false,
      "score": 0,
      "matches": 0,
      "isConnected": true
    },
    {
      "id": "6dbca2da-dcaf-4741-8efc-60de2f9eddd4",
      "socketId": "",
      "name": "rival.evidencia",
      "level": 1,
      "isReady": false,
      "score": 0,
      "matches": 0,
      "isConnected": true
    }
  ],
  "gameState": {
    "status": "waiting",
    "currentRound": 0,
    "totalRounds": 1,
    "cards": [],
    "flippedCards": [],
    "matchedCards": [],
    "currentTurn": null,
    "scores": {}
  }
}
```

Figura 5 - POST /api/rooms/:id/join (200).

Segundo jugador unido a la sala (currentPlayers: 2), base del multijugador en tiempo real.

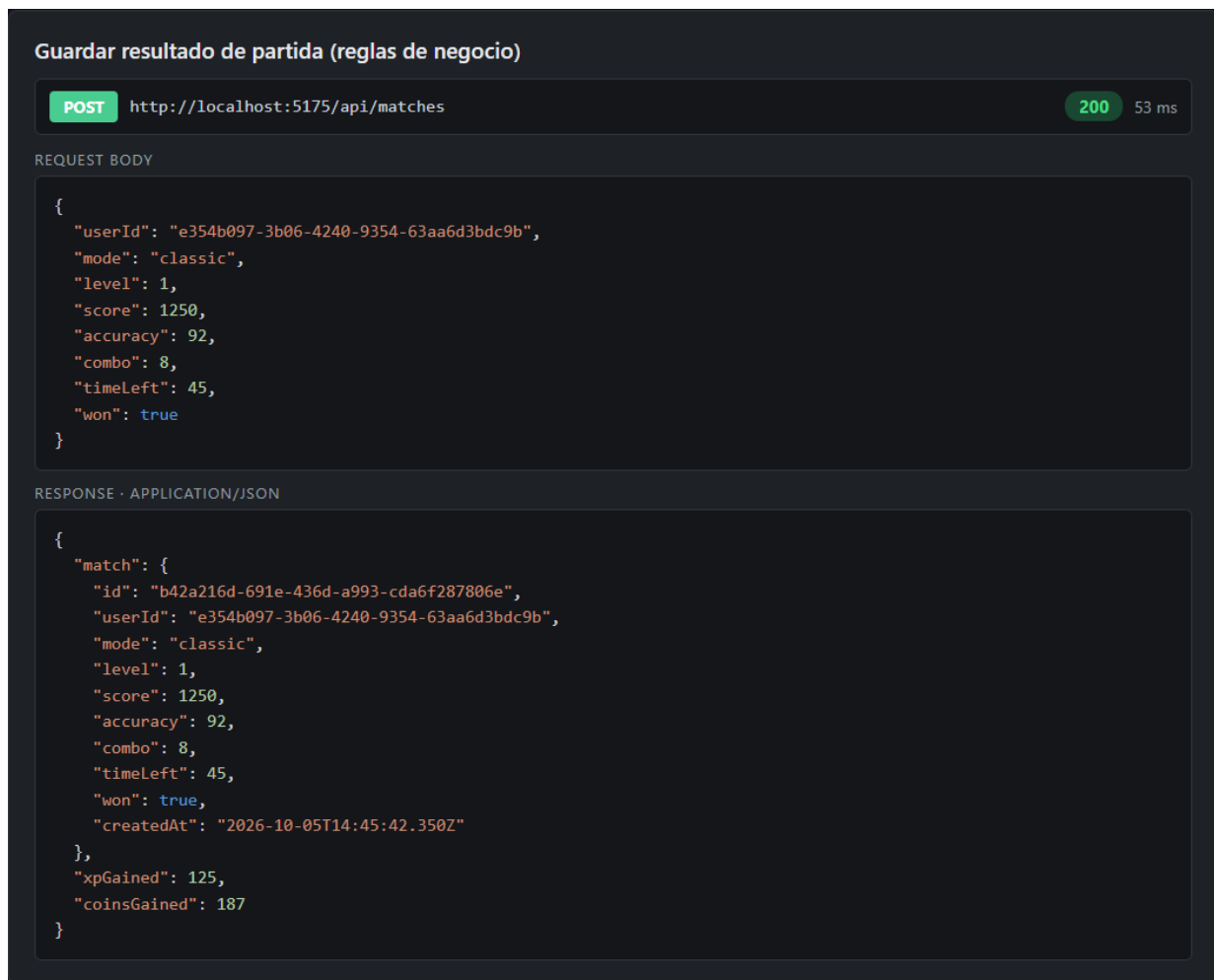


Figura 6 - POST /api/matches (200).

Regla de negocio: al guardar la partida se calculan y devuelven xpGained (125) y coinsGained (187), y se actualizan estadísticas del jugador.

Ranking global por XP

GET http://localhost:5175/api/leaderboard?type=xp&limit=5

200 40 ms

RESPONSE · APPLICATION/JSON

```
{
  "type": "xp",
  "period": "all",
  "leaderboard": [
    {
      "rank": 1,
      "id": "e354b097-3b06-4240-9354-63aa6d3bdc9b",
      "username": "evidencia.backend",
      "email": "evidencia.backend@universidad.edu",
      "level": 1,
      "xp": 125,
      "coins": 687,
      "gems": 50,
      "gamesPlayed": 0,
      "gamesWon": 0,
      "bestScore": 0,
      "totalScore": 0,
      "maxCombo": 0
    },
    {
      "rank": 2,
      "id": "59f90123-c27d-4ba9-9160-5b6d5bdb0d32",
      "username": "kevin",
      "email": "kevinmora2838@gmail.com",
      "level": 1,
      "xp": 0,
      "coins": 500,
      "gems": 50,
      "gamesPlayed": 0,
      "gamesWon": 0,
      "bestScore": 0,
      "totalScore": 0,
      "maxCombo": 0
    },
    {
      "rank": 3,
      "id": "6dbca2da-dcaf-4741-8efc-60de2f9eddd4",
      "username": "rival.evidencia",
      "email": "rival.evidencia@universidad.edu",
      "level": 1,
      "xp": 0,
      "coins": 500,
      "gems": 50,
      "gamesPlayed": 0,
      "gamesWon": 0,
      "bestScore": 0,
      "totalScore": 0,
      "maxCombo": 0
    }
  ],
  "total": 3
}
```

Figura 7 - GET /api/leaderboard?type=xp (200).

Ranking global ordenado por XP con los jugadores de prueba en las primeras posiciones.

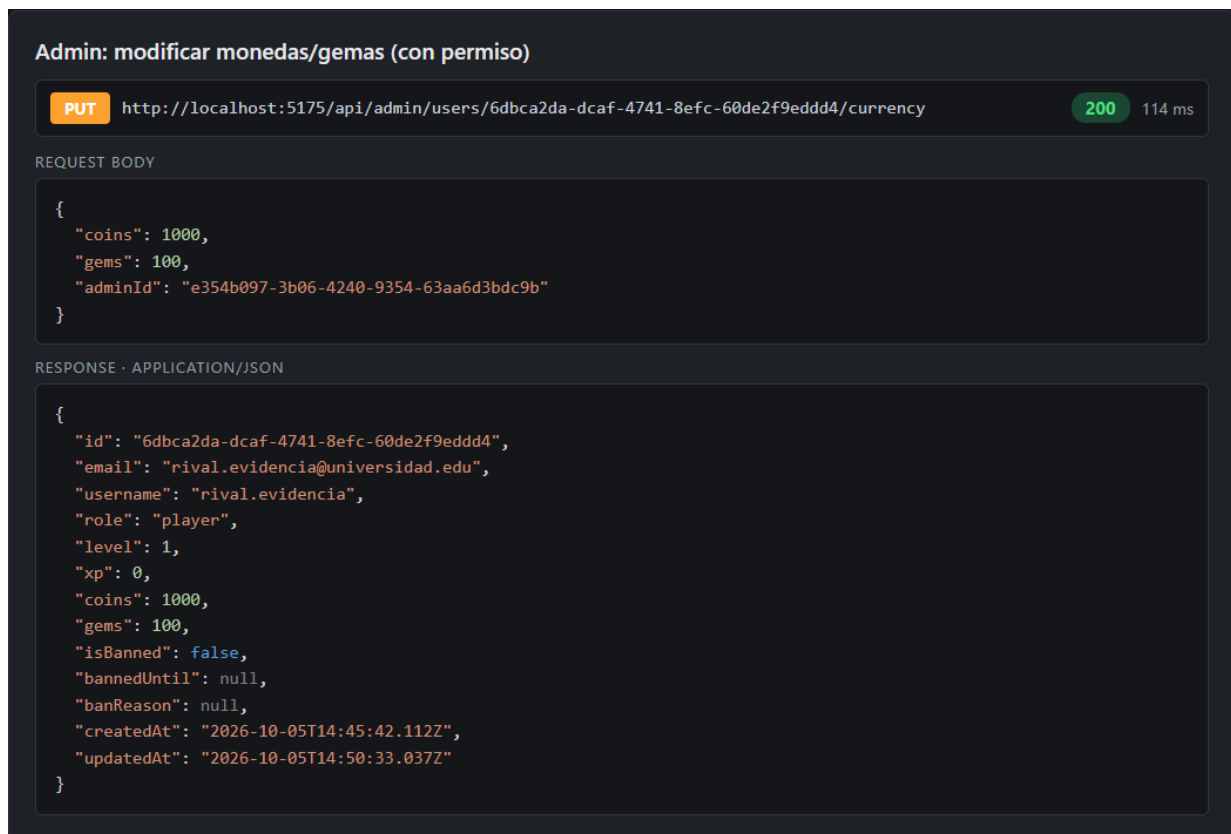


Figura 8 - PUT /api/admin/users/:id/currency (200).

Accion de administrador (con adminId valido): monedas y gemas actualizadas a 1000/100.



Figura 9 - PUT /api/admin/users/:id/currency sin credencial (401).

Seguridad: el middleware requireAdmin rechaza la peticion sin ID de administrador.

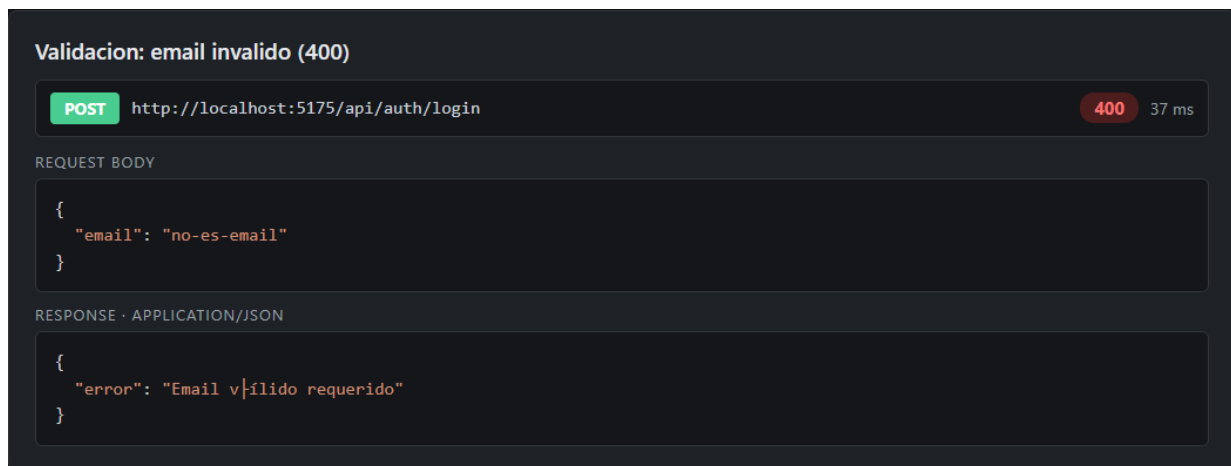


Figura 10 - POST /api/auth/login con email invalido (400).

Validacion de entrada: el controlador rechaza el email malformado antes de tocar la BD.

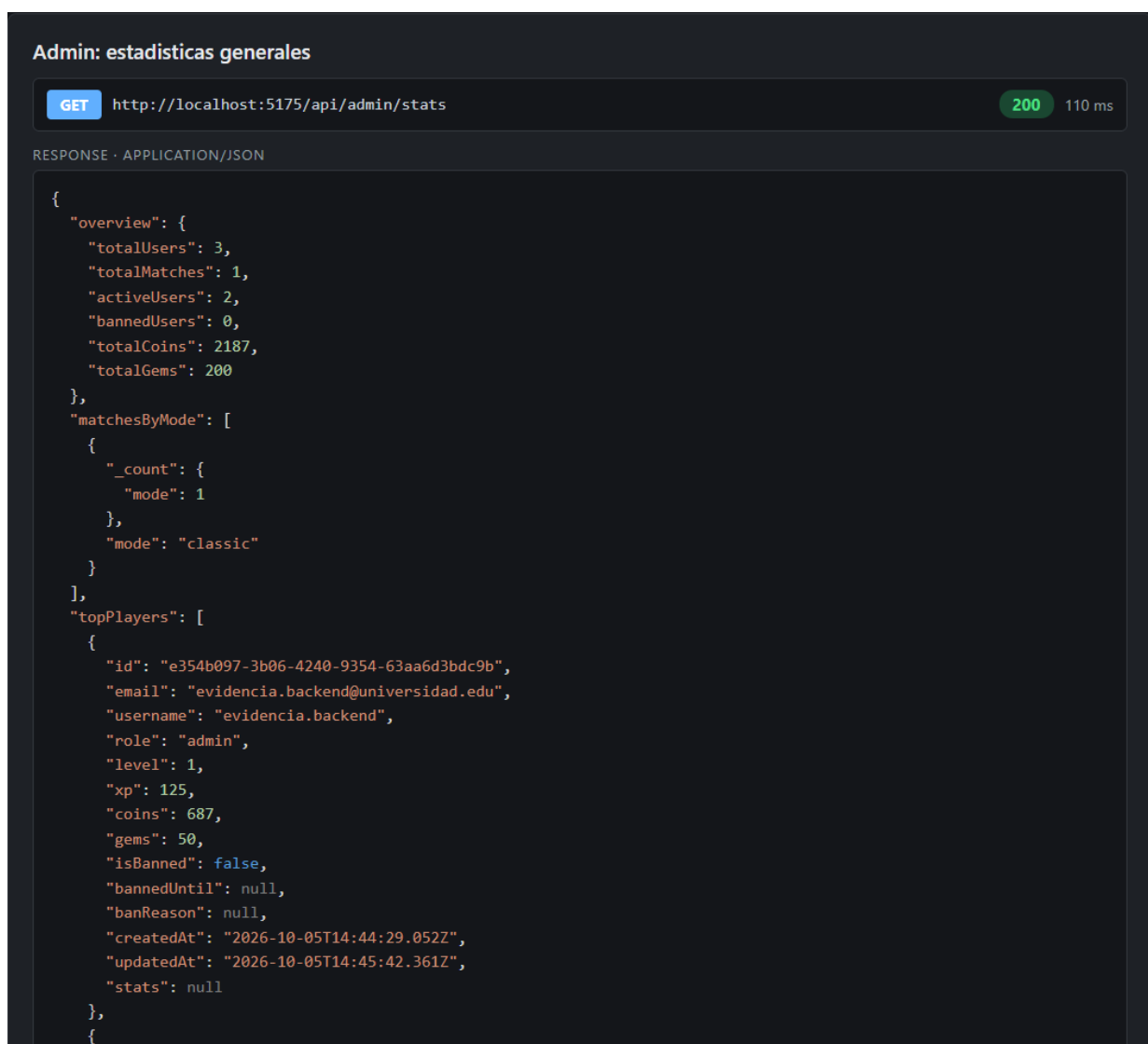


Figura 11a - GET /api/admin/stats (200), parte 1.

Estadísticas generales agregadas: usuarios, partidas, monedas y gemas en circulación.

```

    "createdAt": "2026-10-05T14:44:29.052Z",
    "updatedAt": "2026-10-05T14:45:42.361Z",
    "stats": null
  },
  {
    "id": "59f90123-c27d-4ba9-9160-5b6d5bdb0d32",
    "email": "kevinmora2838@gmail.com",
    "username": "kevin",
    "role": "player",
    "level": 1,
    "xp": 0,
    "coins": 500,
    "gems": 50,
    "isBanned": false,
    "bannedUntil": null,
    "banReason": null,
    "createdAt": "2026-09-14T02:05:17.863Z",
    "updatedAt": "2026-09-14T02:05:17.863Z",
    "stats": null
  },
  {
    "id": "6dbca2da-dcaf-4741-8efc-60de2f9eddd4",
    "email": "rival.evidencia@universidad.edu",
    "username": "rival.evidencia",
    "role": "player",
    "level": 1,
    "xp": 0,
    "coins": 1000,
    "gems": 100,
    "isBanned": false,
    "bannedUntil": null,
    "banReason": null,
    "createdAt": "2026-10-05T14:45:42.112Z",
    "updatedAt": "2026-10-05T14:50:33.037Z",
    "stats": null
  }
]
}

```

Figura 11b - GET /api/admin/stats (200), parte 2.

Detalle por modo y top de jugadores con sus roles y economia, base del panel admin.

5. Conclusion y cumplimiento

Criterio de la entrega	Estado
Repositorio publico con el codigo del backend	Cumple: https://github.com/kevinmora18/memorize (carpeta backend/, rama main)
Fragmentos de logica (controlador, ruta, middleware)	Cumple: routes/auth.ts, AuthController, requireAdmin, AuthService (seccion 3)
Capturas de pruebas a endpoints con JSON	Cumple: 12 capturas reales (health, auth, usuarios, salas, partidas, ranking, admin, seguridad)
Reglas de negocio y seguridad	Cumple: XP/monedas automaticos, baneos, validaciones 400, control de rol 401/403

El backend de Memorize Evolutivo procesa correctamente las reglas de negocio (progresion, economia y multijugador), protege las rutas administrativas por rol, valida las entradas y expone mas de 40 endpoints REST mas 20 eventos Socket.IO, todos verificados en vivo contra PostgreSQL. La capa invisible del sistema esta funcional y lista para sostener al frontend.

Fin de la Evidencia 3 - 30 de septiembre de 2026